Computer Networks – Transport Layer

Assignment 2: Dual-Panel Activity & Transport-Layer Protocol Visualizer

(Extension of Assignment 1 – Application Layer Dashboard)

AI-Assisted Development (Choice of Agentic Platforms & Models)

Course / Subject	Computer Networks – Transport Layer
Development Approach	AI-Agent assisted (student's choice of platform & model)
Prerequisite	Working dual-panel dashboard from Assignment 1 (Application Layer)
Duration	1–2 weeks (recommended)
Submission	Updated source code + AI usage log / artifacts + short demo video + reflection

1. Objective

Extend the dual-panel web dashboard built in Assignment 1 so that, for the same three application-layer activities (**Browsing**, **Mail**, and **Streaming**), the user can also visualize the underlying **Transport-layer protocol exchanges** (primarily **TCP**, with optional **UDP** / **QUIC** where relevant) in real time.

The left panel continues to drive the same user actions. The right panel must now support **two synchronized views** (or a clear toggle / tabbed interface):

- Application-layer view (DNS / HTTP / SMTP) – already implemented in Assignment 1
- Transport-layer view (TCP handshake, data transfer, flow/congestion control signals, connection teardown, and UDP datagrams where applicable)

The goal is to let students see how the same user actions map onto reliable byte-stream delivery (TCP) versus datagram delivery (UDP), and to continue practicing AI-assisted software development.

2. Choice of Agentic AI Platforms & Models

Students remain free to use any reputable agentic AI coding platform or model (same list as Assignment 1). Examples include (but are not limited to):

- Google Antigravity (with Gemini models)
- GitHub Copilot / Copilot Workspace / Copilot Chat
- Claude (Claude Code, Claude.ai, or Anthropic API – Sonnet / Opus)
- Cursor (with Claude, GPT, or Gemini)
- Gemini (Google AI Studio, Gemini Code Assist, or Gemini API)
- OpenAI tools (ChatGPT, Codex, or o-series models)
- Any other reputable agentic coding assistant the student prefers

Students must clearly state in their submission **which platform(s) and model(s)** they used and provide evidence of AI assistance (prompt history, chat logs, agent artifacts, or screenshots).

3. Dashboard Layout (Strict Requirement)

Keep the **exact dual-panel layout** from Assignment 1 (side-by-side on desktop, stacked on smaller screens).

Left Panel – Activity Panel (unchanged in functionality)

- Browsing: URL input + Visit
- Mail: To / Subject / Body + Send
- Streaming: Play / Pause + quality selector
- Activity log and status messages

Right Panel – Protocol Visualization Panel (extended)

Must support two views that stay synchronized with the left-panel action:

1. **Application Layer** view (DNS → HTTP or SMTP) – already present
2. **Transport Layer** view that appears (or can be switched to) immediately when an activity starts

Required Transport-layer visualizations by activity:

Activity	Transport Protocols to Visualize
Browsing	TCP 3-way handshake → data segments (Seq/ACK numbers) → connection teardown (FIN/ACK)
Mail	TCP 3-way handshake → SMTP data transfer over the TCP stream → teardown
Streaming	Primary: TCP (or optional QUIC) for manifest + segments; optional UDP illustration for comparison

For each Transport-layer step the visualization must show:

- Exact messages / flags (SYN, SYN-ACK, ACK, PSH, FIN, RST, etc.)
- Sequence numbers, acknowledgement numbers, window size, and flags
- Direction (client → server or server → client)
- Timing / sequence indicators
- Key fields highlighted (Seq, Ack, Win, Flags, Length)
- Simple animation or progressive reveal (arrows, timeline, or state-machine style)
- Ability to **pause**, **step forward/backward**, or **replay**

Users must be able to switch between (or view side-by-side) the Application-layer and Transport-layer visualizations while keeping both synchronized with the left-panel action.

4. Technical & AI Requirements

- The project must be developed with substantial assistance from an agentic AI coding tool of the student's choice (see Section 2).
- Build on the existing Assignment 1 codebase. Students are expected to use the AI for architecture of the dual-view right panel, accurate TCP state machine and message generation, live synchronization, and debugging / iterating.
- Preferred technology stack: Python + FastAPI or Flask + HTML/JS. Streamlit or Gradio are also acceptable. A pure frontend (React/Vue) is allowed.
- Simulation of protocols is fully acceptable (real network sockets are optional extra credit). Visualizations must remain accurate with respect to real TCP/UDP behavior.
- Students must retain and submit evidence of AI usage (prompts, responses, agent artifacts, or screenshots).

5. Deliverables

1. Working extended dashboard (complete source code + clear instructions to run it).
2. AI usage log / artifacts (prompt history, chat exports, agent records, or screenshots) clearly showing the platform and model used.
3. Short demo video (2–4 minutes) or a set of screenshots showing all three activities with **both** Application-layer and Transport-layer visualizations updating in parallel.
4. Reflection document (1–2 pages) covering:

- Which AI platform(s) and model(s) were chosen and why
- How the Application-layer and Transport-layer views stay synchronized with the left panel
- What the AI got wrong (especially TCP sequence/ACK logic or state transitions) and how you corrected it
- Key differences observed between the Transport-layer flows for Browsing (short request-response), Mail (longer conversational stream), and Streaming (multiple segment transfers)

6. Evaluation Criteria

Criterion	Weight
Correct dual-panel layout + live synchronization of both App-layer and Transport-layer views	30%
Accuracy of TCP (and optional UDP/QUIC) message visualization, sequence/ACK numbers, and state transitions	30%

Effective use of chosen AI platform/model (prompt quality, iteration, evidence)	20%
Code quality, UI polish, and reflection document	20%

7. Suggested Starter Prompt (Adapt to Your Chosen AI Tool)

You may adapt the following prompt for Antigravity, Cursor, Claude, Copilot, Gemini, or any other tool:

```
Extend the existing dual-panel dashboard (Assignment 1) so that the right panel now supports
both Application-layer and Transport-layer visualizations.

Keep the left panel unchanged (Browsing / Mail / Streaming).

For the right panel:
• Add a clear toggle or tabs: “Application Layer” | “Transport Layer”
• When an activity starts, both views must update in parallel and stay synchronized.

Transport-layer requirements:
• Browsing → TCP 3-way handshake, data segments with Seq/Ack/Flags, connection teardown
• Mail → same TCP lifecycle carrying the SMTP conversation
• Streaming → TCP (or optional QUIC) for manifest + media segments; optionally show a UDP comparison

Show exact TCP messages (SYN, SYN-ACK, ACK, FIN, etc.), sequence & acknowledgement numbers,
window size, direction, and timing. Support pause / next / previous / replay.

Reuse as much of the existing code as possible and keep the two panels synchronized.
```

8. Important Notes for Students

- Focus on **correctness of TCP state transitions, sequence/ACK numbering, and message ordering** more than fancy graphics.
- Clearly document the AI platform and model you used; this is part of the evaluation.
- If the AI generates incorrect TCP behavior (wrong flags, missing handshake steps, incorrect sequence numbers), you must correct it and explain the correction in your reflection.
- Extra credit ideas: real socket-based TCP servers, basic congestion-window illustration, comparison of persistent vs non-persistent TCP connections, QUIC handshake visualization, or side-by-side TCP vs UDP for the streaming case.

This assignment continues the bridge from user action → application-layer messages → transport-layer delivery, while giving further hands-on experience with modern AI-agent development using a platform of your choice.

Good luck!